



SYED SUALEH ABBAS 10303

MOHSIN MUJTABA 10016

Lab 3: Implementing Jump and Return in RISC V Assembly Language

Objectives

This lab introduces RISC-V assembly instructions related to control transfer, specifically jump and return operations. Students will implement and analyze jump-based program flow using Visual Studio Code by completing a series of hands-on tasks designed to strengthen their understanding of subroutines and control flow in RISC-V.

Introduction

In this lab we will learn about:

- RISC-V Jump and Return instructions
- Handle the stack

Function calls

Functions, or procedures as they are sometimes called, are an important part of programming. They help avoid repeating the same code all over our programs. A function call would take the program flow to the start of the function code. That code will be executed, and a return from-function instruction would take the program flow back to where originally the function was called from and continue the execution from the next instruction after the function call.

Like all instruction sets, RISC-V provides special instructions to enable function calls and return from functions.

Category	Instruction	Assembly	Operation	Description
Unconditional branch	Jump and link	jal x1, 100	x1 = PC+4; go to PC+100	PC-relative procedure call
	Jump and link register	jalr x1, 100(x5)	x1 = PC+4; go to x5+100	Procedure return; indirect call

Table 3.1: RISC-V Unconditional Branch Instructions

You can think of a procedure like a spy who leaves with a secret plan, acquires resources, performs the task, covers his or her tracks, and then returns to the point of origin with the desired result. Nothing else should be perturbed once the mission is complete. Moreover, a spy operates on only a “need to know” basis, so the spy can’t make assumptions about the spymaster. Similarly, in the execution of a procedure, the program



must follow these six steps:

- Put parameters in a place where the procedure can access them.
- Transfer control to the procedure.

- Acquire the storage resources needed for the procedure.
- Perform the desired task.
- Put the result value in a place where the calling program can access it.
- Return control to the point of origin, since a procedure can be called from several points in a program

```
jal    x1, func_name
jalr   x0, 0(x1)
```

Listing 1: Function instructions

Listing 1 lists the jal and jalr instructions used for function calls and returns from functions respectively in RISC-V.

Executing jal instruction would make the program flow jump to func name label in the code while at the same time storing the address of the next instruction PC+4 in the register x1. The return address needs to be stored so that the program flow can go back to where the function call was made to continue execution from there onwards once it has finished executing the function code. Since a function can be called from multiple locations in a program, without saving the return address we will not have a clue as to where to return to. The jalr instruction is used to return from functions. The jump target address is provided in a register. The example shown in Listing 1, it will jump to the address stored in the register x1. Combining this with jal instruction we can see how the two instructions work in tandem to provide the functionality of calling a function and returning from it.

RISC-V Registers			
Register	ABI Name	Description	Saved by
x0	zero	zero register (hardwired zero)	
x1	ra	return address	Caller
x2	sp	stack pointer	Callee
x3	gp	global pointer	
x4	tp	thread pointer	
x5–7	t0–2	temporary registers	Caller
x8	s0 / fp	saved register / frame pointer	Callee
x9	s1	saved register	Callee
x10–11	a0–1	function arguments / return values	Caller
x12–17	a2–7	function arguments	Caller
x18–27	s2–11	saved registers	Callee
x28–31	t3–6	temporary registers	Caller

Table 3.2: RISC-V Registers

Another issue associated with function calls is passing the parameters and returning the result. As can be seen from Table 3.2, the RISC-V convention reserves the registers x10-x17 for passing parameters to a function when calling it. The caller code would fill in these registers the parameters needed by a function before calling it. Once the function is called, it will access its parameters from these registers, use them, and then return the result in x10 before calling the jalr instruction which would take the control flow to the caller who can then access the result of the function call by reading x10.

In case the registers x10-x17 contains some values, which will still be used by the caller code after the function call, the caller code needs to store these values as well as its own return address x1 on the stack before overwriting these values with the ones needed by the callee. See section 2.8 in the textbook.

Environment Calls

The Venus simulator supports some system calls, which it calls Environment Calls, as well. See the url <https://github.com/kvakil/venus/wiki/Environmental-Calls>. To execute an environment call we have to put its number in the register a0(x10), any argument it might need in a1(x11) and then call the ecall instruction. The simulator will perform the system call specified in the a0(x10) register.

The RISC-V environment calls can be used to print values from registers or memory in the Venus Terminal. For example, one of the supported system calls is print int whose number is 1 and which will print any value stored in the register a1(x11) in the terminal as an integer.

```
addi a0 x0 1      # print int  ecall  
addi a1 x0 42     # integer 42  
ecall
```

Listing 2: Print integer

The code in listing 2, taken from the above url, would print the integer 42 on the Venus Terminal Window.

Task 1:

Write a code for the function sum shown in listing 3. It takes two integers a and b as arguments and returns their sum. You should also write code that calls this function i.e. it sets up the two arguments a and b in registers x10 and x11 and then calls sum(). After calling sum(), it then retrieves the returned result from the register x10. It can display the result using the ecall instruction.



```
int sum(int a, int b)
{
    return a+b;
}
```

```
addi x10,x0,12
addi x11,x0,12
jal x1,sum #calls the function named sum
addi x11,x10,0
li x10,1
ecall
j exit
sum:
    add x10,x11,x10
    jalr x0,0(x1)
exit:
```

Listing 3: Sum

Code

```
.text
.globl

main:
addi x10, x0, 12
addi x11, x0, 12
jal x1, sum
add x11, x10, x0
li x10, 1
ecall
j exit
sum:
add x10, x11, x10
jalr x0, 0(x1)

exit:
```

Output

```
task1.s
1  .text
2  .globl
3
4  main:
5  addi x10, x0, 12
6  addi x11, x0, 12
7  jal x1, sum
8  add x11, x10, x0
9  li x10, 1
10 ecall
11 j exit
12 sum:
13 add x10, x11, x10
14 jalr x0, 0(x1)
15
16 exit:
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS

Starting program C:\Users\Administrator\OneDrive - Habib University\Sem 4\CA\Labs\Lab03\Task1.s

Exited with error code 1

Stop program execution!

Starting program C:\Users\Administrator\OneDrive - Habib University\Sem 4\CA\Labs\Lab03\Task1.s

24Exited with error code 1

Stop program execution!

**Task 2:**

Have a look at a C procedure:

```
long long int leaf_example (long long int g, long long int h, long long int i,  
long long int j)  
{  
    long long int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

What is the RISC-V assembly code for the above procedure?

The parameter variables g, h, i, and j correspond to the argument registers x10, x11, x12, and x13 and f corresponds to register x20. In addition, the program uses two temporary registers x18 and x19. Thus, we need to save three registers x18, x19, and x20 onto the stack.

Listing 4: Leaf Procedure

**Code**

```
.text
.globl
main:
li x10, 10 #g
li x11, 6 #h
li x12, 4 #i
li x13, 9 #j
jal x1, stack
jal x2, load
j end

stack:
    addi sp, sp, -8
    sw x5, 4(sp)
    sw x4, 0(sp)
    add x5, x10, x11
    add x4, x12, x13
    sub x20, x5, x4 #f
    lw x4, 0(sp)
    lw x5, 4(sp)
    addi sp, sp, 8
    jalr x0, 0(x1)

load:
    li x10, 1
    add x11, x20, x0
    ecall
    jalr x0, 0(x2)
end:
j end
```



Output

The screenshot displays the Visual Studio Code (VS Code) interface with the assembly code for `Task2.s` open in the editor. The code is as follows:

```
1 .file "Task2.s"
2
3 main:
4     li x10, 10 #g
5     li x11, 6 #h
6     li x12, 4 #i
7     li x13, 9 #j
8     jal x1, stack
9     jal x2, load
10    j end
11
12 stack:
13     addi sp, sp, -8
14     sw x5, 4(sp)
15     sw x4, 0(sp)
16     add x5, x10, x11
17     add x4, x12, x13
18     sub x20, x5, x4 #f
19     lw x4, 0(sp)
20     lw x5, 4(sp)
21     addi sp, sp, 8
22     jalr x0, 0(x1)
23
24 load:
25     li x10, 1
26     add x11, x20, x0
27     ecall
28     jalr x0, 0(x2)
29
30 end:
31     j end
32
```

The left sidebar shows the **VARIABLES** panel with the following values:

Register	Value
x01 (ra)	0x00000014
x02 (sp)	0x00000018
x03 (gp)	0x10000000
x04 (tp)	0x00000000
x05 (t0)	0x00000000
x06 (t1)	0x00000000
x07 (t2)	0x00000000
x08 (s0)	0x00000000
x09 (s1)	0x00000000
x10 (a0)	0x00000001
x11 (a1)	0x00000003
x12 (a2)	0x00000004
x13 (a3)	0x00000009
x14 (a4)	0x00000000
x15 (a5)	0x00000000
x16 (a6)	0x00000000
x17 (a7)	0x00000000
x18 (s2)	0x00000000
x19 (s3)	0x00000000
x20 (s4)	0x00000003
x21 (s5)	0x00000000
x22 (s6)	0x00000000
x23 (s7)	0x00000000

The **CALL STACK** panel shows the current call stack:

Function	Address
jal x0 0	Task2.s 30

The **TERMINAL** panel shows the following output:

```
Stop program execution!
-----
Starting program C:\Users\Administrator\OneDrive - Habib University\Sem 4\CA\Labs\Lab03\Task2.s
3
```



```
void swap (long long int v[ ], size_t k)
```

```
{  
    long long int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

What is the RISC-V assembly code for the above procedure?

The parameter variables v[] and k correspond to the argument registers x10 and x11.

Listing 5: Swap Procedure

Task 3:

The following C procedure swaps two array elements (or two locations in memory)

Code

```
.text  
.globl main  
  
main:  
    li t0, 0x200    # base address  
    li x25, 5       # number of elements = 5  
    li x22, 0       # i = 0  
    j loop1  
  
loop1:  
    bge x22, x25, swap  
    slli x10, x22, 2 # offset = i * 4  
    add t1, t0, x10  
    addi x11, x22, 1 # value = i + 1 (1 to 5)  
    sw x11, 0(t1)  
    addi x22, x22, 1  
    j loop1  
  
swap:  
    li x14, 0       # k = 0  
    slli x16, x14, 2  
    add x17, t0, x16  
    addi x18, x17, 4  
    lw x19, 0(x17)  
    lw x20, 0(x18)  
  
    sw x20, 0(x17)   # A[k] = A[k+1]  
    sw x19, 0(x18)  
end:  
    j end
```



Output

BEFORE SWAPING:

0x00000220	00	00	00	00
0x0000021C	00	00	00	00
0x00000218	00	00	00	00
0x00000214	00	00	00	00
0x00000210	05	00	00	00
0x0000020C	04	00	00	00
0x00000208	03	00	00	00
0x00000204	02	00	00	00
0x00000200	01	00	00	00
0x000001FC	00	00	00	00
0x000001F8	00	00	00	00

AFTER SWAPING:-

0x00000228	00	00	00	00
0x00000224	00	00	00	00
0x00000220	00	00	00	00
0x0000021C	00	00	00	00
0x00000218	00	00	00	00
0x00000214	00	00	00	00
0x00000210	05	00	00	00
0x0000020C	04	00	00	00
0x00000208	03	00	00	00
0x00000204	01	00	00	00
0x00000200	02	00	00	00
0x000001FC	00	00	00	00
0x000001F8	00	00	00	00

Task 4:

The procedure strcpy copies string y to string x using the null byte termination convention of C:

```
void strcpy (char x [], char y[])

{

    size_t i;

    i = 0;

    while ((x[i] = y[i]) != '\0') /* copy & test byte */

        i += 1;

}
```

What is the RISC-V assembly code?

Code

```
.data
src: .asciiz "Hello World"
dst: .space 10
.text
.globl main
main:
    la t0, src
    la t1, dst
    j copy
copy:
    lb t2, 0(t0)
    li t3, 32
    beq t3,t2, end #stop at the ascii value of empty space
    sb t2, 0(t1)
    beq t2, zero, end # stop at '\0'
    addi t0, t0, 1
    addi t1, t1, 1
    j copy
end:
    j end
```



Output

Address	+0	+1	+2	+3
0x10000018	0x00	0x00	0x00	0x00
0x10000014	0x00	0x00	0x00	0x00
0x10000010	o	0x00	0x00	0x00
0x1000000C	H	e	l	l
0x10000008	r	l	d	0x00
0x10000004	o		W	o
0x10000000	H	e	l	l
0x0FFFFFFC	0x00	0x00	0x00	0x00
0x0FFFFFF8	0x00	0x00	0x00	0x00
0x0FFFFFF4	0x00	0x00	0x00	0x00
0x0FFFFFF0	0x00	0x00	0x00	0x00
0x0FFFFFEC	0x00	0x00	0x00	0x00
0x0FFFFFE8	0x00	0x00	0x00	0x00

Address: Up Down

Jump to: -- choose -- ▾

Display Format: ASCII ▾

Bytes per Row: 4 ▾



Assessment Rubric

Lab 3: Implementing Jump and Return Introductions in RISC V Assembly Language

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/10	/05
	Task 2	/10	/10
	Task 3	/10	/10
	Task 4	/15	/10
Total Points: 100		/45	/35
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.